S Y L L A B U S

Harness Engineering

Skills, Subagents, Hooks & Production Systems

A First-Principles Curriculum for What Will Last

For Bangladeshi CSE Graduates | 12 Weeks | 12 Modules | 6 Capstone Projects

Version 3.0 — March 2026

"The model is commodity. The harness is moat."

Teaches patterns, not frameworks. Primitives, not APIs. What compounds, not what trends.

Prepared by

Abul Kalam Faruk

Design Philosophy

This syllabus was designed by asking one question of every topic: will this pattern still matter when the models are 10x better? Frameworks rewrite. APIs change. Model rankings shuffle quarterly. But certain primitives compound forever. This curriculum teaches those primitives.

What We Cut and Why

Framework-specific APIs (CrewAI, AutoGen, LangChain internals) — they rewrite constantly; patterns transfer, syntax doesn't. RAG as a standalone discipline — it's absorbed into context engineering. A2A protocol deep-dive — important to know, not ready to build on. Detailed model comparisons — obsolete in 3 months. Low-code tools (n8n) — learnable in an afternoon. Specific orchestrators (Gas Town, Conductor) — implementations of patterns we do teach.

What We Kept and Why — The Permanent Stack

How LLMs work (physics, not fashion). Programmatic Tool Calling / PTC (code as the natural action primitive, now GA). MCP (settled standard, 97M+ monthly downloads). The agent loop (perception → reasoning → action → observation). Context engineering (what the LLM sees determines everything). Skills, Subagents, Hooks, Plugins (the extension system that's already cross-tool: Claude Code, Codex, Gemini CLI, Cursor). Architectural constraints via mechanical enforcement. Feedback loops via tests and linters. Agentic commerce (Stripe ACP, SPTs — agents that transact are fundamentally more valuable). Evaluation and observability. The Bitter Lesson: build lightweight, design for deletion.

Program Structure

Phase	Focus	Weeks
I: Primitives	LLM internals, tool calling, MCP, the agent loop	1–3
II: The Extension System	Context engineering, Skills, Subagents, Hooks, Plugins	4–7
III: Harness Discipline	Constraints, feedback loops, workflow design, harness types	8–10
IV: Production	Evaluation, deployment, domain harnesses, capstone	11–12

Prerequisites

- Proficiency in Python (OOP, async/await, type hints)
- Familiarity with REST APIs, JSON, Git, CLI
- Basic ML concepts (not expert-level)

Phase I: Primitives (Weeks 1–3)

The atoms that everything else is built from. These don't change when models improve — they become more important.

Module 1: How LLMs Work & How to Control Them	
Key Topics	• Transformer architecture: tokenization, embeddings, self-attention, positional encoding — why this matters for harness design • Inference: temperature, top-p, KV caching, context window mechanics • Prompt engineering as a subset of context engineering: zero-shot, few-shot, chain-of-thought, structured outputs • The 40% context utilization limit: why overloading context makes agents worse, not better (Dex Horthy's research) • Model evaluation methodology: how to evaluate models for YOUR task, not benchmark leaderboards • Token economics: cost per task, when to use large vs. small models, routing strategies
Tools	Any frontier API (Anthropic, OpenAI, Google), LM Studio, Ollama for local models
Deliverable	**Build a model evaluation harness that tests any LLM on your own task suite and outputs cost/quality/latency tradeoffs**

Module 2: Tool Calling: From JSON to Programmatic (PTC)	
Key Topics	• Traditional tool calling: JSON schema definition, parameter validation, return handling — the legacy path • Why traditional calling is becoming obsolete: each invocation = full inference pass, intermediate results pollute context • Programmatic Tool Calling (PTC): agents write code that calls tools in a sandbox — now GA as of Feb 2026 with Sonnet 4.6 • PTC results: 37% token reduction, improved accuracy (25.6% → 28.5% knowledge retrieval, 46.5% → 51.2% GIA benchmarks) • The core insight: LLMs are trained on billions of lines of code, not JSON schemas. Code is their natural action primitive. • Tool Search Tool: agents search a tool library on demand instead of loading all definitions upfront (85% token reduction) • Building tools for PTC: designing functions that work in sandboxed code execution environments • The tool selection problem: why Vercel removed 80% of tools and got better results — less is more • PTC + MCP together: code execution calling MCP tools, keeping intermediate results out of context
Tools	Anthropic PTC (code_execution tool), Tool Search Tool, MCP servers, custom Python tools

Deliverable	**Build a PTC-powered research agent: write code that orchestrates 5+ MCP tools in a single execution block. Compare token usage and accuracy against traditional tool calling on the same task.**

Module 3: MCP: The Settled Protocol Standard	
Key Topics	• Why MCP won: 97M+ monthly SDK downloads, adopted by every major provider, governed by Linux Foundation • Architecture: client-server model, JSON-RPC 2.0, transports (stdio, SSE, HTTP Streaming) • Building MCP servers: exposing tools, resources, and prompts to any AI client • Building MCP clients: connecting any agent to any MCP-compatible tool • Runtime tool discovery vs. hardcoded connections — why this changes everything • MCP security: authentication, authorization, sandboxing, permission scopes • The protocol landscape: A2A for agent-to-agent (early but important), WebMCP for browsers (emerging)
Tools	MCP Python SDK, MCP TypeScript SDK, Claude Desktop, any MCP-compatible client
Deliverable	**Build 3 MCP servers (database, web API, file system) and a client that discovers and uses them dynamically**

Phase II: The Extension System (Weeks 4–7)

This is the layer that's becoming the universal interface between humans and AI agents. Skills, Subagents, Hooks, and Plugins are already cross-tool (Claude Code, Codex, Gemini CLI, Cursor) and cross-domain. 9,000+ plugins exist. 713+ skills in universal SKILL.md format. This is the packaging system for agent expertise, and it will only grow.

Module 4: Context Engineering & AGENTS.md	
Key Topics	• The hierarchy: prompt engineering ⊂ context engineering ⊂ harness engineering • "From the agent's point of view, anything it can't access in-context doesn't exist" (OpenAI) • AGENTS.md / CLAUDE.md as living infrastructure: every agent mistake becomes a prevention rule • The "table of contents, not encyclopedia" pattern: small entry file pointing to deeper structured docs • Three-tier context architecture: Tier 1 (always loaded), Tier 2 (skill-specific, on demand), Tier 3 (persistent knowledge base) • Context compaction: summarization, filesystem-as-memory, state offloading, sub-agent isolation • Dynamic context: giving agents access to logs, metrics, browser state, runtime signals • Why context engineering is the highest-leverage skill: LangChain went from rank 30 to rank 5 by changing only context, not the model
Tools	AGENTS.md convention, CLAUDE.md, structured docs/ directories, Markdown
Deliverable	**Engineer a complete context system for a real project: AGENTS.md, structured docs, tiered loading. Measure agent task success before vs. after.**

Module 5: Skills: Packaging Expert Knowledge for Agents	
Key Topics	• What skills are: on-demand context providers that auto-activate based on description matching • SKILL.md anatomy: name, description, trigger conditions, the knowledge payload • Skills vs. AGENTS.md: always-on context (AGENTS.md) vs. on-demand expertise (Skills) — context budget management • The universal SKILL.md format: works across Claude Code, Codex, Gemini CLI, Cursor — write once, use everywhere • Building domain skills: frontend design, code review, security auditing, document generation, data analysis • Skills for research: scanning Reddit/X/web for topics, competitive analysis, market intelligence • Skills as harness components: the frontend-design skill is just a harness

	for UI generation • The Smithery catalog and awesome-claude-code: 713+ community skills, what makes a good skill • Lazy loading and the context budget: “Smaller system prompt + load skill X when needed” (Harrison Chase)
Tools	SKILL.md format, Claude Code /skills, Smithery catalog, awesome-skills.com
Deliverable	**Build 5 production skills: a Bangla document processor, a code review skill, a security audit skill, a financial analysis skill, and a research skill. Publish to a marketplace.**

Module 6: Subagents & Agent Teams: Delegation as Architecture	
Key Topics	• Subagents: isolated workers that do a job, return a result, never talk to each other. Parent relays information. • Agent Teams: collaborative squads that share a task list and message each other directly • When to use subagents (independent tasks) vs. agent teams (coordinated work) • The context isolation principle: heavy lifting in separate context windows keeps the main session clean • Subagent configuration: system prompts, tool permissions, model selection per agent • Parallel execution patterns: 10 subagents scanning a codebase simultaneously • Planning/execution separation: dedicated planning agent → execution agents (the OPENDEV dual-agent pattern) • Practical patterns: bootstrap-repo (parallel scanners), design review workflows, competitive analysis teams • Constraint: subagents cannot spawn subagents — designing flat delegation hierarchies
Tools	Claude Code subagents and Agent Teams, Codex multi-agent, custom orchestration scripts
Deliverable	**Build a multi-agent system: a lead agent that delegates to 4 specialized subagents (researcher, analyst, coder, reviewer), with an Agent Team variant that coordinates directly**

Module 7: Hooks, Plugins & the Distribution Layer	
Key Topics	• Hooks: deterministic lifecycle control — PreToolUse, PostToolUse, PermissionRequest, session start/end • Hook patterns: auto-lint on every file write, block dangerous commands, push notifications, aggregate context on session start • Security hooks: 49+ packs protecting git, databases, cloud providers from destructive commands • Plugins: the packaging format that bundles Skills + Subagents + Hooks + MCP Servers into distributable units • Plugin anatomy: manifest (plugin.json), namespaced skills (/plugin-name:skill), installer scripts • Standalone (.claude/) vs. Plugin: personal iteration first, package for sharing when stable • Marketplace ecosystem: Anthropic official, community marketplaces (ComposioHQ, buildwithclaude.com), 9,000+ plugins • Plugin hygiene: context budget impact, lazy loading for MCP but not yet for skills, the “5 well-chosen > 20 mediocre” rule • Building your own marketplace: structuring a GitHub repo as a plugin source
Tools	Claude Code hooks, plugins, /plugin command, marketplace infrastructure,

	GitHub repos
Deliverable	**Build and publish a complete plugin: 3 skills + 2 subagents + security hooks + an MCP server, packaged with manifest and installer. Deploy to a marketplace.**

Phase III: Harness Discipline (Weeks 8–10)

The universal engineering principles that apply regardless of which harness type you're building: coding CLI, browser, desktop, runtime, or domain-specific. These are the patterns that made OpenAI's Codex team produce 1M lines with zero human-written code, and that made Meta pay $2B for Manus.

Module 8: Constraints & Feedback Loops	
Key Topics	• The core paradox: stricter constraints produce more reliable output. Constraining the solution space is a feature. • Architectural boundaries enforced mechanically: dependency layers, custom linters, structural tests • Custom linter error messages as agent remediation instructions — tooling that teaches while it works • Test coverage as agent infrastructure: without tests, agents cannot verify their own work • Agent-to-agent review: implementation agent + reviewer agent = mutual oversight loop • CI/CD as harness: pre-commit hooks for instant feedback (not just CI gates in the cloud) • Garbage collection agents: periodic sweeps for doc drift, architectural violations, dead patterns • The self-repair loop: agent fails → identify missing capability → encode fix into repo → agent improves permanently • "Golden principles": opinionated mechanical rules that keep systems legible for future agent runs
Tools	Custom linters (ESLint/Ruff custom rules), ArchUnit, pre-commit hooks, GitHub Actions CI, pytest/vitest
Deliverable	**Build a constraint + feedback system from scratch: architectural linters with remediation messages, test harness, agent-review pipeline, and a garbage collection agent. Measure drift over 50 agent commits.**

Module 9: Workflow Design & The Bitter Lesson	
Key Topics	• The subtraction principle: Vercel removed 80% of tools and agents got better. Manus rewrote 5 times, each time simpler. • The Bitter Lesson applied to harnesses: every model upgrade makes some harness logic obsolete. Design for deletion. • Permission systems: least-privilege, scoped tools, human-in-the-loop gates for sensitive operations • Task isolation: git worktrees, sandboxed VMs, filesystem scoping to prevent cross-contamination • Session management: running 3–4 concurrent agents, dead-time utilization, notifications for when agents need input • Harness types as deployment contexts (not separate disciplines): coding

	CLI, IDE, browser, desktop, runtime, domain • The harness-as-operating-system analogy: model = CPU, context = RAM, harness = OS. Build the OS, swap the CPU freely. • Re-architecture readiness: LangChain rebuilt 4 times in one year. Manus 5 times in 6 months. This is normal, not failure. • Instruction fade-out: why agents drift after 100+ steps and how context durability engineering solves it
Tools	Git worktrees, Docker, permission configs, tmux/screen, session management tools
Deliverable	**Design a workflow system for a team of 3 engineers: permission gates, task isolation, parallel agents, session handoff protocol. Test it over a 1-week sprint.**

Module 10: Evaluation, Observability & Security	
Key Topics	• Agent evaluation as its own discipline: non-deterministic systems need non-traditional testing • LLM-as-judge: designing eval prompts, calibrating scores, component-level evals (retrieval, planning, execution) • Scenario-based testing: simulated environments, adversarial inputs, edge cases, regression suites • Observability as table stakes: 89% of production teams trace agent execution step-by-step • Tracing: step-by-step visibility into reasoning chains, tool calls, token usage, cost per task • Dogfooding as evaluation: real-world failure capture beats synthetic benchmarks • Testing harnesses across models: what works for Claude may need harness iteration for Codex (LangChain's finding) • Security surface: prompt injection (direct + indirect), data exfiltration, permission escalation • Defense: input sanitization, output filtering, privilege separation, OWASP LLM Top 10
Tools	TruLens, LangSmith, Langfuse, Arize Phoenix, OpenTelemetry, custom eval frameworks
Deliverable	**Build an evaluation + observability pipeline: 50+ test scenarios, full tracing dashboard, security red-team exercise with documented mitigations**

Phase IV: Production (Weeks 11–12)

Deployment, domain-specific harness design for Bangladesh use cases, and the capstone.

Module 11: Domain Harnesses & Agentic Commerce	
Key Topics	• Harnesses as the new service templates: shared golden paths per domain, team-contributed and maintained • Agentic commerce: Stripe's Agentic Commerce Suite — the economic infrastructure layer for agents that transact • Shared Payment Tokens (SPTs): the new payment primitive — scoped by seller/time/amount, agents pay without exposing credentials • Agentic Commerce Protocol (ACP): open-source spec for agent-to-business commerce, implementable as REST or MCP server • Stripe's agent toolkit and MCP server (mcp.stripe.com): adding payments to any agent via MCP • Building commerce-capable agents: product discovery, checkout flows, fraud detection (Radar), order management • Domain skills and plugins for Bangladesh: fintech (bKash/Nagad SPT integration), RMG supply chain, Bangla document processing, e-governance • Browser harness patterns: automating government portals, form-filling agents with session persistence • Deployment: Docker, CI/CD, staged rollouts, cost optimization (model routing, caching, context budgets) • The brownfield problem: incremental harness adoption (AGENTS.md first → hooks → linters → full harness)
Tools	Stripe Agent Toolkit, Stripe MCP Server, ACP, SPTs, Docker, AWS/GCP, domain-specific MCP servers
Deliverable	**Build an agentic commerce agent: product discovery via MCP, checkout via Stripe ACP/SPTs, with a domain skill for a Bangladesh vertical. Deploy with CI/CD and cost monitoring.**

Module 12: Capstone: Ship a Production Harness	
Key Topics	• Choose a domain: fintech, agri-tech, e-governance, supply chain, edtech, healthcare • Build a complete harness: AGENTS.md, Skills (3+), Subagents (2+), Hooks, at least 1 MCP server, architectural constraints • Package as a Plugin publishable to a marketplace • Evaluation pipeline: scenario tests, component evals, before/after harness performance data • Deploy: containerized, observable, monitored, with CI/CD • Security audit documentation • Final presentation: live demo + harness architecture + measurable impact metrics + what you'd delete when models improve

Tools	Full stack from all modules
Deliverable	**A production-grade harness solving a real Bangladesh problem: packaged as a Plugin, deployed, evaluated, with measurable before/after data proving the harness effect**

Essential Reading

Harness Engineering Canon

- OpenAI: “Harness engineering: leveraging Codex in an agent-first world” (Feb 2026)
- Mitchell Hashimoto: “My AI Adoption Journey” (Feb 2026) — the six stages
- Martin Fowler / Birgitta Böckeler: “Harness Engineering” (Feb 2026)
- Phil Schmid: “The importance of Agent Harness in 2026”
- Harrison Chase (LangChain CEO): VentureBeat podcast on harness evolution (Mar 2026)
- Antonino Ingargiola: “The Rise of the Agent Harness” — migration from coding CLIs to general infrastructure
- Dean Blank: “A Mental Model for Claude Code: Skills, Subagents, and Plugins” (Mar 2026)
- OPENDEV paper (arXiv:2603.05344): dual-agent architecture, lazy tool discovery
- Addy Osmani: “Top AI Coding Trends for 2026” — Ralph loops, Tasks, Skills ecosystem

Protocol & Ecosystem

- MCP Specification (modelcontextprotocol.io)
- Anthropic: “Advanced Tool Use” — Programmatic Tool Calling, Tool Search Tool, dynamic discovery (anthropic.com/engineering)
- Anthropic: PTC Cookbook (platform.claude.com/cookbook) — hands-on PTC implementation
- Stripe: Agentic Commerce Suite, ACP, Shared Payment Tokens, Agent Toolkit (stripe.com/blog/agentic-commerce-suite)
- Stripe MCP Server (mcp.stripe.com) and Agent Toolkit (github.com/stripe/ai)
- AGENTS.md convention (agents.md)
- Claude Code docs: Plugins, Skills, Subagents, Hooks (code.claude.com/docs)
- awesome-skills.com — 123+ curated skills and plugins
- buildwithclaude.com — plugin marketplace (117 skills, 175 subagents, 28 commands)
- Anthropic Agentic Coding Trends Report 2026
- LangChain State of Agent Engineering Report 2026

Career Pathways

What this program prepares you for:

- **Harness Engineer:** Design constraints, feedback loops, and lifecycle systems. The defining role of 2026 — Meta paid $2B for this capability.

- **Skills & Plugin Developer:** Build and distribute agent expertise as Skills and Plugins. The npm/pip ecosystem for AI agents.
- **AI Agent Engineer:** Build agents with harness-first thinking across coding, browser, desktop, and runtime contexts.
- **AI Platform / Runtime Engineer:** Persistent agent infrastructure: session management, sandbox execution, multi-tenant isolation.
- **MCP Protocol Engineer:** Build and maintain tool interoperability layers.
- **AgentOps Engineer:** Deploy, monitor, evaluate, and scale harnessed systems.
- **Domain AI Engineer:** Vertical harnesses for fintech, supply chain, healthcare, e-governance with domain-specific constraints.

Assessment

Component	Weight	Format
Module Projects (1–11)	40%	Code + harness artifacts
Capstone (Module 12)	30%	Live demo + before/after metrics
Skills & Plugin Portfolio	20%	Published skills, plugins, marketplace contributions
Peer Review	10%	Agent trace reviews + feedback

Why Bangladesh, Why Now

94% of engineering leaders globally report critical gaps in agentic AI expertise. Meta paid $2B for a harness, not a model. The skills and plugin ecosystem already has 9,000+ entries and is growing weekly. Bangladeshi engineers who can author production Skills, build domain Plugins, and design harnesses for fintech, RMG, and e-governance will be among the most globally competitive technical professionals — because they'll own the layer that actually makes AI work.

Everything in this syllabus was chosen because it compounds. AGENTS.md updates prevent classes of failures forever. Skills package expertise once and deploy everywhere. Plugins distribute to teams instantly. Linters teach every future agent session. These are not features that expire with the next model release. They are the permanent infrastructure of the agentic era.